

Week 2 Day 2 — LangChain: Tools, Chains, Memory & Framework Agents

Overview: Day 1 implemented a raw ReAct loop with explicit HTTP message payloads. Day 2 refactors this into LangChain's component ecosystem, adding LCEL pipelines, multi-turn session memory, external catalog querying, Pydantic structured output, and resilient tool error handling.

1. Concept Mapping & LCEL Mechanics

Concept	Day 1 (Raw Python)	Day 2 (LangChain Abstraction)
LLM Wrapper	genai.Client + generate_content	ChatGoogleGenerativeAI (implements Runnable interface)
Tool Registry	TOOL_SCHEMAS dict + Python fns	@tool / StructuredTool (docstring -> function calling schema)
Agent Loop	while-loop in agent.py (manual dispatch)	create_tool_calling_agent + AgentExecutor (automated ReAct)
Memory	Explicit messages list you append	ConversationBufferMemory / RunnableWithMessageHistory

Under the hood of LCEL (|): LangChain overloads Python's bitwise OR (`__or__`) operator across `Runnable` classes to build a `RunnableSequence`. When executing `prompt | llm | StrOutputParser()`, the output of each link is piped as input to the next, automatically orchestrating streaming, async execution, batching, and OpenTelemetry callback tracing.

2. Tool Docstrings & External Data Source

Tool docstrings function directly as prompt instructions: LangChain inspects function type hints and extracts docstrings into the model's tool schema. Reused Day 1's safe AST calculator and `get_weather`, and added `lookup_product_price` to read a local JSON data source (`data/products.json`).

3. Annotated Agent Reasoning Trace (ReAct Verification)

Phase	Agent Execution Step & Event Trace	Subsystem / State
REASON	Model inspects user query + system prompt + tool descriptions; decides tool calls.	AgentExecutor Plan
ACT	Invoking: <code>get_weather({'city': 'Karachi'})</code> -> returns 33 C, hot, humid.	Tool Invocation
ACT	Invoking: <code>get_weather({'city': 'London'})</code> -> returns 12 C, overcast, light rain.	Tool Invocation
OBSERVE	Observations injected into <code>agent_scratchpad</code> as <code>tool_result</code> / <code>FunctionResponse</code> .	Scratchpad Update
ACT	Invoking: <code>calculator({'expression': '33 - 12'})</code> -> returns 21.	Tool Invocation
FINAL	Model synthesizes: 'Karachi is warmer than London by 21 degrees Celsius.'	Terminal Plain Text

4. Conversation Memory & Pydantic Structured Output

3-Turn Memory Scenario: Tested multi-turn dependency: (1) *Find price of Pro (\$49)* -> (2) *Compare to Enterprise (\$149)* -> (3) *Which to recommend to a budget client?* `ConversationBufferMemory` preserves past turns in `chat_history`, allowing the model to resolve 'Which one' without re-querying. **Structured Output:** Forced the final recommendation into a typed `ProductRecommendation` Pydantic schema (`recommended_product: str, price_usd: float, rationale: str, budget_friendly: bool`) via `llm.with_structured_output()` for deterministic downstream consumption.

5. Error Handling & Architectural Trade-Offs (Raw vs LangChain)

Graceful Error Recovery: In `broken_lookup`, throwing `ToolException` configured with `handle_tool_error=True` intercepts runtime crashes and converts the exception into a synthetic observation, allowing the model to acknowledge upstream timeouts rather than terminating the process. **What LangChain Made Easier:** Drastically reduced boilerplate: declarative LCEL piping, turnkey memory managers, automated JSON Schema generation from docstrings/type hints, and built-in structured schema parsing. **Abstraction Leakiness & 'Magic':** (1) *Package churn:* Legacy `AgentExecutor` was relegated to `langchain_classic`. (2) *Opaque state:* Prompt scratchpad formatting and Gemini thought signatures are hidden; subtle token/state mismatches fail as remote 400 Bad Requests rather than readable Python exceptions. (3) *Payload obscurity:* `verbose=True` surfaces tool I/O but masks raw HTTP requests.